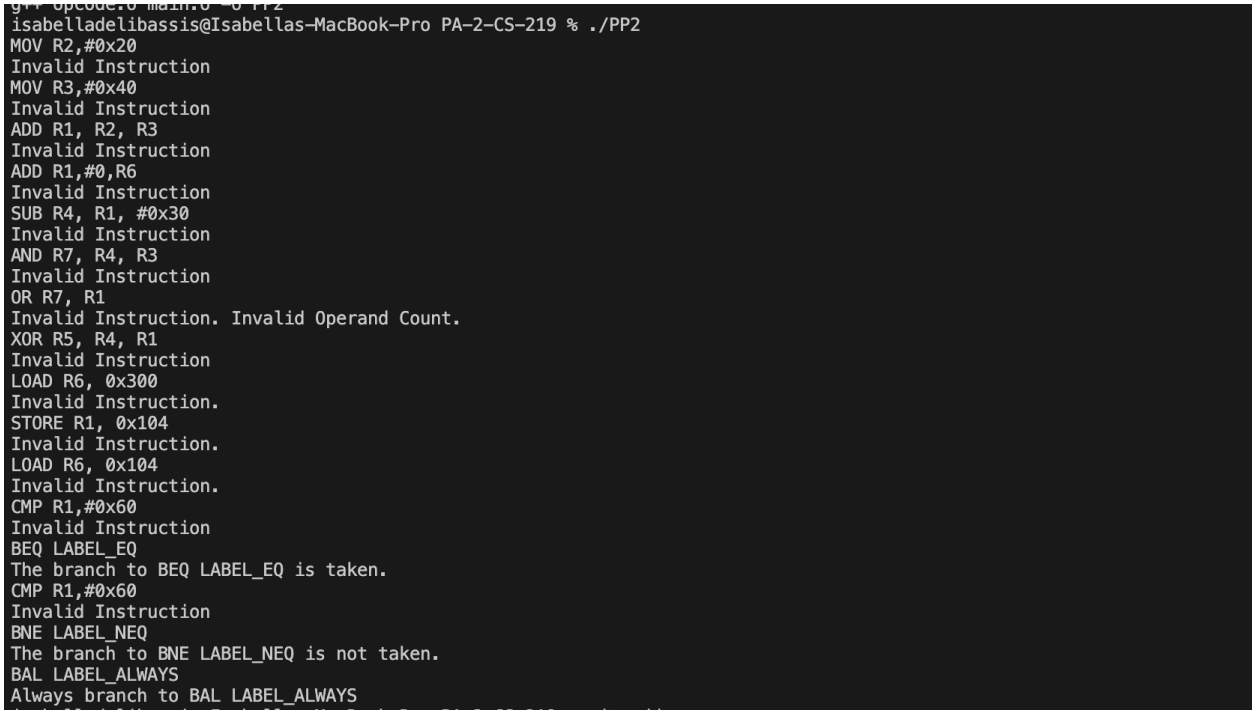


In order to run the code, it runs in C++ code on VS. The executable name is PP2. To make the code, you just type in “make”, to make clean the code is type in “make clean”, and to run the code, you must type in “./PP2”. The code’s working process is that in the main.cpp file, we are reading from the input.txt file, so we will use an “ifstream to read the entire file, and use getline to read each line one at a time. Afterwards, we will then read between the commas and read the operation name, and the operands given from the line. After getting out of the while loop to retrieve the operands, we will then see which operation we will need to perform.

If the operation name is: “ADD”, “SUB”, “AND”, “OR”, or “XOR”, then we will run the arithmetic and logical operations function. In this function, we will first check the number of operands that are involved from the line. If the number of operands does not equal 3, then we will print out “Invalid Instruction, Invalid Operand Count.” and break out of the function. If it does equal 3, then we will check then if the registers that are being used have values within them. If not, then we will print out Invalid Instruction, but if yes then we will store the values in the two registers based on the line (ranging from 0 to 7). Based on the operation name, the first parameter will determine what the result will be.

If the operation name is MOV, then we will run the MOV function. In this function, we will first check the number of operands that are involved from the getline. If the number of operands does not equal 2, then we will print out “Invalid Instruction, Invalid Operand Count.” and break out of the function. If it does equal 2, then we will check then if the register destination that is being asked for is within range, and also check the operand and make sure that it has a value, and if either are incorrect, then we will break out of the function. If not, then we will continue and store the second operand in the list into the first operand.

If the operation name is CMP, then we will run the CMP function. In this function, we will first check the number of operands that are involved from the getline. If the number of operands does not equal 2, then we will print out “Invalid Instruction.” and break out of the function. If it

does equal 2, then we will check then if the register destination that is being asked for is within range, and also check the operand and make sure that it has a value and if either are incorrect, then we will break out of the function. If not, then we will continue and store the results of the two operands in r1 and r2. Then we will make a *result* variable and subtract r2 from r1. And if the result is equal to 0, then the zero variable will be equal to 1 since the *result* is equal to 0. However, if the result does not equal 0, then the zero variable will be 0.

If the operation name is: LOAD or STORE, then we will run the dataTransfer function. In this function, we will first check the number of operands that are involved from the getline. If the number of operands does not equal 2, then we will print out “Invalid Instruction.” and break out of the function. If it does equal 2, then we will check then if the register destination that is being asked for is within range, and also check the operand and make sure that it has a value and if either are incorrect, then we will break out of the function. Next, depending on the operation name, it will determine what the result will be.

If the operation name is: BEQ, BNE, or BAL, then we will run the branch function. First we will check if there is only one label name, if not then we will print out “Invalid Instruction.”. If there is only one label name, then we will check the operation name. If the operation name is BEQ, then we will then check if the zero variable is 1, then that means that both of the previous operands from the CMP are equal to each other which we will print out that “The branch to LABELNAME is taken.”, but if the zero variable is 0, then that means that the operands are not equal to each other “The branch to LABELNAME is not taken.”. If the operation name is BNE, then we will check if the zero variable is 0, then that means that both of the previous operands from the CMP are not equal to each other and means that “The branch to LabelName is taken.” will be printed out since the two are not equal to each other, but if z = 1, then that means that they are equal to each other, meaning that “The branch to LabelName is not taken.” will be printed out since we are asking if the two operands are not equal to each other. Lastly, if the operation name is BAL, then regardless of the zero variable, this branch will be taken.

For all of the operation names, we will print out the line that was inputted from the input.txt file. If the operation name is: ADD, SUB, AND, OR, XOR, LOAD, STORE, or CMP, we will also print out the register list as well as the memory address list. If there is no value for a register, then it will automatically print out “0x0”, and if there is no memory address for in any of the indexes, then it will print out “___”.